

CSC 630 2.0 — DATA SCIENCE, BUSINESS INTELLIGENCE & BIG DATA ANALYTICS

Employee Attrition Prediction

Using diagnostic and predictive analytics to identify why employees leave —and flag who is likely to leave next.

B.M.H.C.C.K. Hapugaskumbura : GS/COMP/315

MSc in Computer Science — Final Assignment

Dataset: IBM HR Analytics Employee Attrition & Performance (Kaggle)

Why This Project Matters

Employee attrition drives up recruitment cost, erases institutional knowledge, and hurts productivity. This project applies the full data science lifecycle to a real HR dataset to find out why people leave — and to flag who is likely to leave next.



Business Problem

Understand why employees leave the organization, and predict which current employees are most likely to leave next.



Business Objective

Give HR the ability to design targeted retention strategies — pay adjustments, career paths, workload changes — before resignations happen, not after.



Analytical Approach

Follow the BI pipeline: Data → Preprocessing → Exploration → Mining (Feature Engineering + Modeling) → Presentation → Decision, applying descriptive, diagnostic and predictive analytics.

Dataset Overview

1,470

Employee records

35 → 31

Attributes after cleanup

1

Binary target: Attrition

IBM / Kaggle

HR Analytics source data

Feature Mix

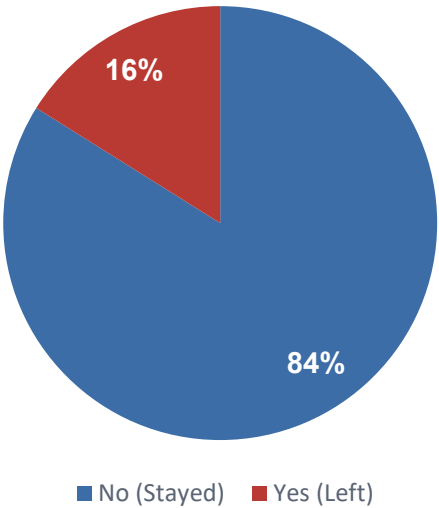
- 14

Continuous numerical
Age, MonthlyIncome, YearsAtCompany ...
- 9

Ordinal categorical
JobSatisfaction, WorkLifeBalance (1–5) ...
- 8

Nominal categorical
Department, JobRole, MaritalStatus ...

Target Variable Balance



83.9% stayed vs. 16.1% left — a clear class imbalance that shapes every modeling decision later on.

Getting the Data Analysis-Ready

1

Missing values

Checked all 35 columns — zero missing values found. No imputation required.

2

Duplicate rows

All 1,470 records confirmed unique — zero duplicates detected.

3

Non-informative columns

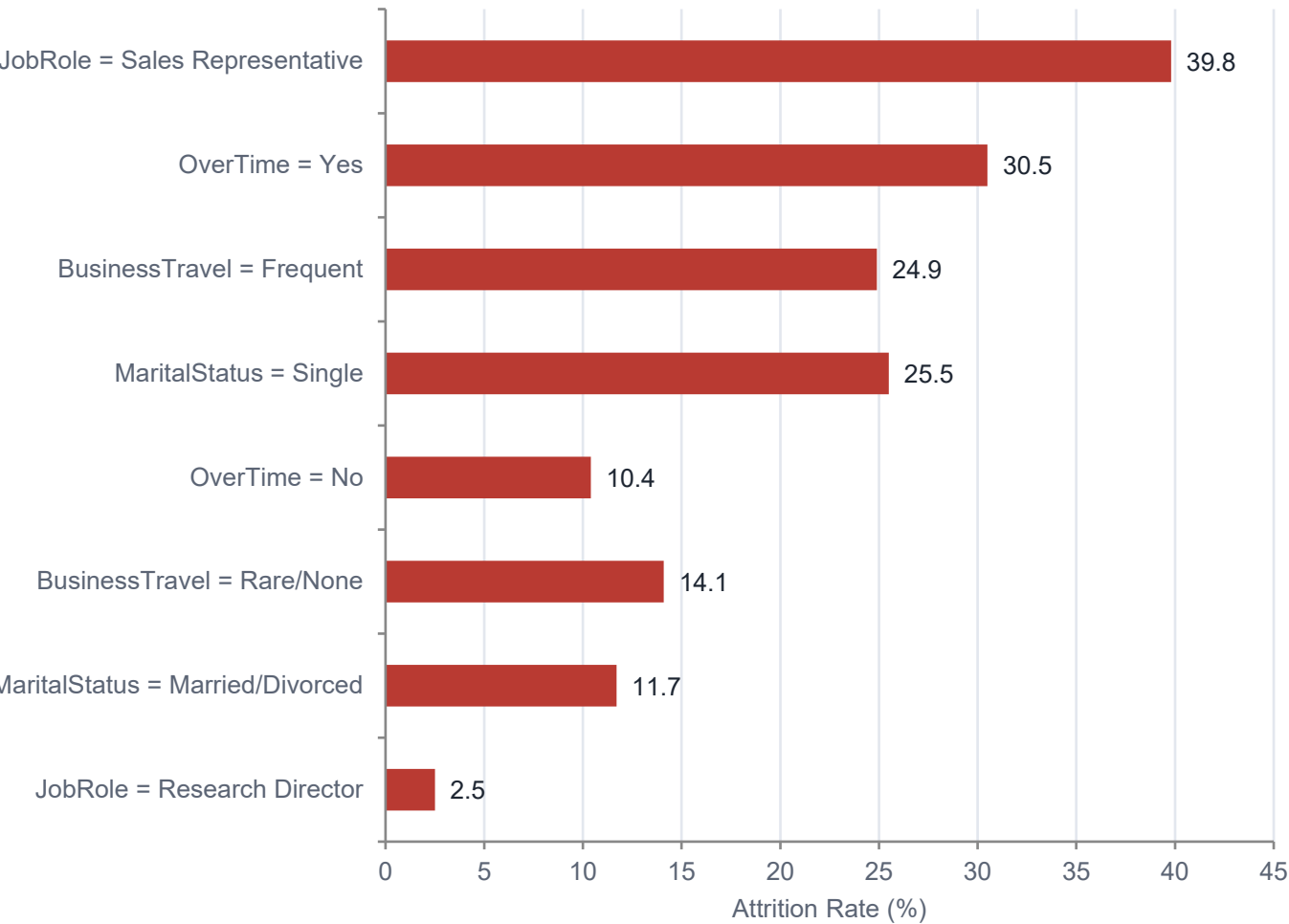
EmployeeCount, Over18, StandardHours each had only 1 unique value — dropped, along with EmployeeNumber.

4

Categorical encoding

Compared One-Hot vs. Frequency encoding on a baseline Random Forest — One-Hot selected for interpretability.

Why Does Attrition Happen?



Ranked Root Causes

Job Role

16x gap: Sales Reps (39.8%) vs. Research Directors (2.5%)

Overtime

~3x higher attrition (30.5% vs. 10.4%)

Business Travel

Frequent travelers attrite ~1.8x more (24.9% vs. 14.1%)

Marital Status

Single employees attrite more (25.5% vs. 11.7%)

Turning Diagnosis Into Model Signal

Diagnostic findings directly informed three new engineered features, expanding the feature set from 31 to 55 columns after encoding.

EDITED

AgeGroup

Binned continuous Age into 5 interpretable ranges (18–25 ... 56–65) for business-friendly reporting.

ADDED

HighStrain

Flags employees who work overtime OR travel frequently — combining the two strongest bivariate risk factors into one signal.

MULTI-LAYER

IncomePerYearWorked

$\text{MonthlyIncome} \div \text{TotalWorkingYears}$ — an interaction feature capturing pay relative to experience, distinct from either source column.

Model Training Approach



Why stratified split + scaling?

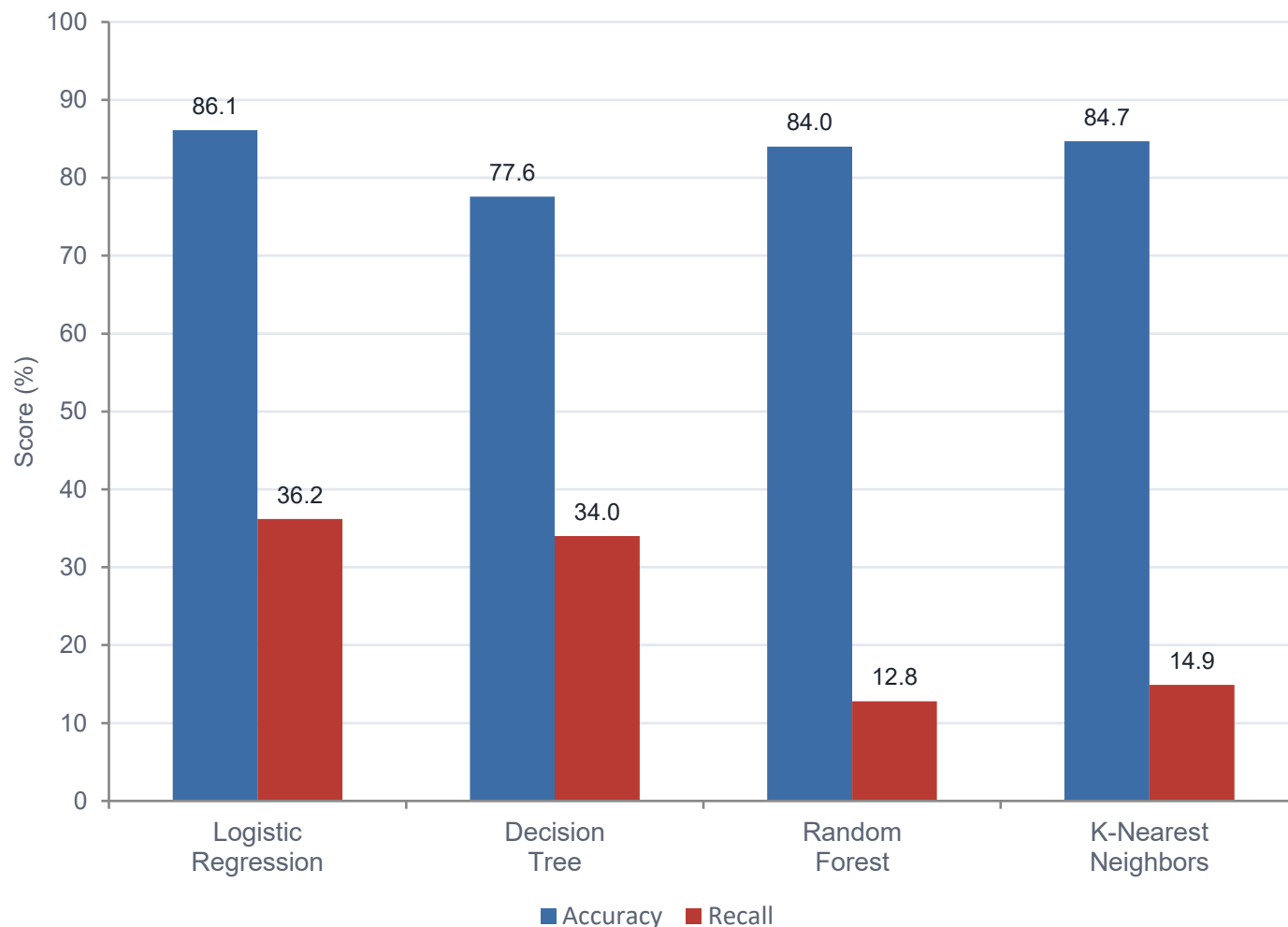
- Stratified 80/20 split preserves the 84% / 16% class balance in both train (1,176) and test (294) sets, avoiding a skewed evaluation.
- StandardScaler standardizes all 55 features to mean 0 / std 1 — essential for Logistic Regression and KNN, which are sensitive to feature magnitude.

Four Models Compared

- **Logistic Regression** Simple, interpretable linear baseline
- **Decision Tree** Captures non-linear rules, prone to overfitting
- **Random Forest** Ensemble of trees, reduces overfitting
- **K-Nearest Neighbors** Distance-based, sensitive to scale & dimensionality

Key challenge: class imbalance (84/16) biases naive models toward always predicting “No” — handled via stratified sampling and, later, class-weighted optimization.

Accuracy Can Be Misleading



The Paradox

Random Forest and KNN post the highest accuracy (84.0% / 84.7%) but the lowest recall (12.8% / 14.9%) — they miss most employees who actually leave, because both are biased toward the majority “No” class.

Logistic Regression wins on recall

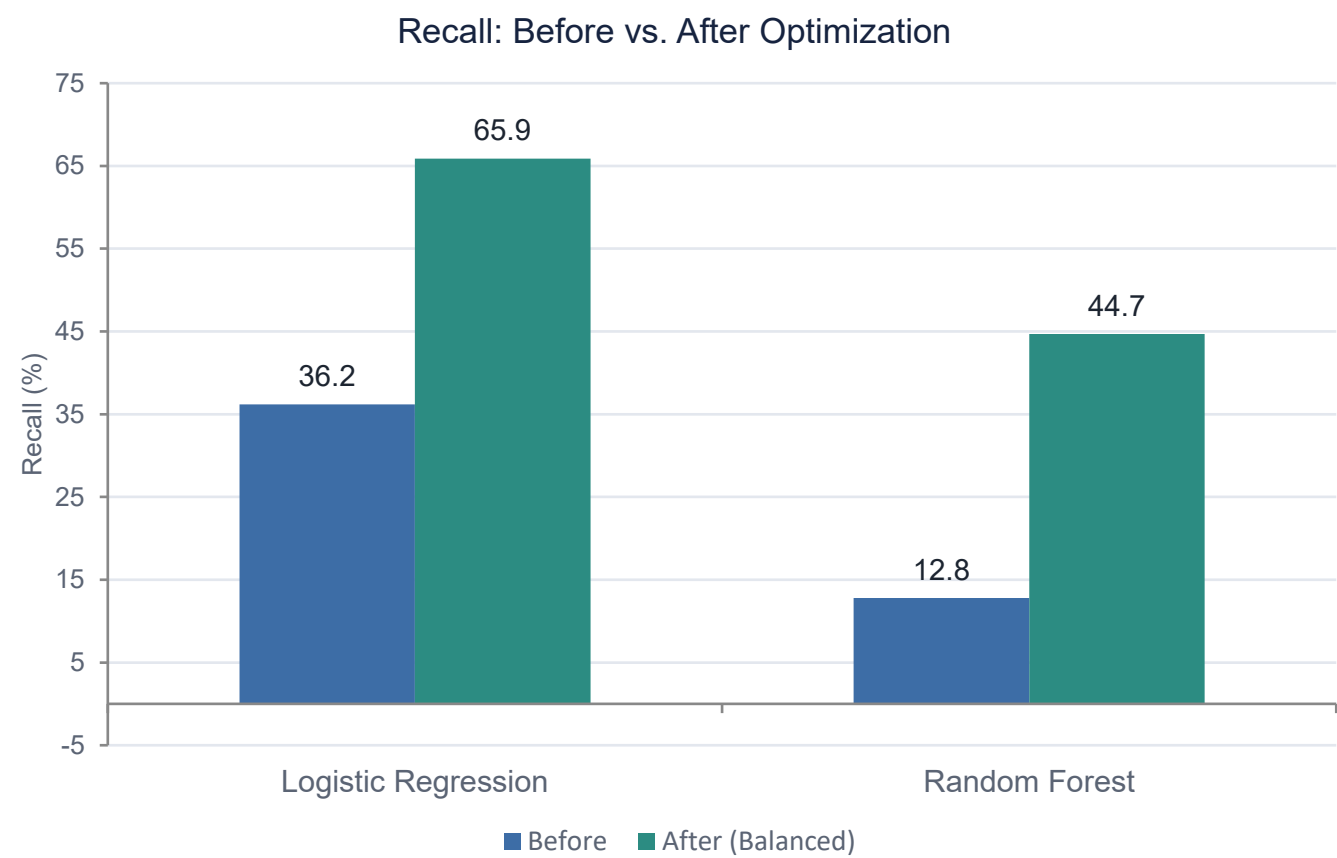
36.2% recall, 86.1% accuracy, and the smallest train–test gap — good generalization, no overfitting.

Decision Tree overfits badly

100% train accuracy collapses to 77.6% on test — a 22.4-point gap, the largest of any model.

Fixing Recall With Class Weighting

Retraining with `class_weight='balanced'` penalizes misclassifying the minority “left” class more heavily — directly targeting the recall problem.



65.9%

Logistic Regression recall (up from 36.2%) — AUC holds at 0.811

44.7%

Random Forest recall (up from 12.8%) — F1 more than doubles to 45.7%

Trade-off: precision drops (e.g. 60.7% → 36.9% for Logistic Regression) — acceptable here, since missing an at-risk employee costs HR more than a false alarm.

From Model to Action

Final model: Logistic Regression (Balanced) — 65.9% recall, 81.1% AUC, interpretable coefficients for HR communication.

1	Target overtime reduction	Redistribute workload / hire additional staff in high-overtime departments.
2	Reassess travel policy	Offer flexibility or added compensation for frequent business travelers.
3	Prioritize Sales Reps	40% attrition — the highest of any role — needs targeted career & mentorship programs.
4	Review low-income pay	Focus on long-tenure, comparatively low-paid employees flagged by IncomePerYearWorked.
5	Deploy as early-warning system	Generate a monthly attrition-risk score per employee for proactive HR outreach.

Inside the Model: Class-Weighted Optimization

The snippet below (from the project notebook) retrains Logistic Regression and Random Forest with `class_weight='balanced'` to fix the recall problem identified during evaluation.

```
attrition_model.py

1  from sklearn.linear_model import LogisticRegression
2  from sklearn.ensemble import RandomForestClassifier
3
4  optimized_models = {
5      'Logistic Regression (Balanced)': LogisticRegression(
6          max_iter=1000, class_weight='balanced', random_state=42),
7      'Random Forest (Balanced)': RandomForestClassifier(
8          class_weight='balanced', random_state=42, max_depth=8)
9  }
10
11 for name, model in optimized_models.items():
12     model.fit(X_train_scaled, y_train)
13     preds = model.predict(X_test_scaled)
14     proba = model.predict_proba(X_test_scaled)[: , 1]
```

What This Code Does

1

`class_weight='balanced'`

Automatically reweights the loss function so the minority class (employees who left) is penalized more heavily during training.

2

`.fit()` → `.predict()` → `.predict_proba()`

Trains on the scaled feature matrix, then generates both hard predictions and class probabilities (needed for the AUC metric).

3

Loop over both models

The same training/evaluation logic runs for Logistic Regression and Random Forest, keeping the comparison fair and reproducible.

Result

Logistic Regression recall: 36.2% → 65.9%

AUC holds steady at 0.811

Limitations & Conclusion

Limitations

- Single point-in-time snapshot — no longitudinal trend data.
- Public sample dataset — real deployment requires retraining on the organization's actual current data.
- Precision remains moderate (36.9%) even after optimization — risk scores should guide prioritization, not serve as definitive predictions.

Conclusion

This project carried the full data science lifecycle — business understanding, preprocessing, EDA, feature engineering, predictive modeling, and optimization — through to a real HR attrition problem.

Overtime, business travel, job role, and compensation emerged as the primary drivers of attrition. The optimized Logistic Regression model now identifies nearly two-thirds of at-risk employees, turning diagnostic insight into an actionable, evidence-based retention tool for HR.

Thank You

Questions & Discussion

References

- IBM HR Analytics Employee Attrition & Performance dataset — Kaggle
- scikit-learn documentation — scikit-learn.org
- pandas documentation — pandas.pydata.org